

[Edit](#)

Refactor Python Code by Using Copilot

Challenge Overview

Understand the scenario

You are a Developer at Hexelo, a company that needs to improve code quality and maintainability using AI-assisted refactoring. In this Challenge Lab, you will refactor repetitive loops into smaller, reusable functions using GitHub Copilot. Next, you will enhance code clarity by adding type hints and comments suggested by Copilot. Finally, you will improve code efficiency by implementing Copilot's suggestions for simplification and generalization.

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSCContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Refactor code into reusable functions

Hints Enabled

No Yes

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using `Passw0rd!` as the password.
- Enter your GitHub Username, Email Address, and Password into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

- Sign in to `Visual Studio Code` with your GitHub account as `<GithubUN>` by using `<GithubPW>` as the password.

 Expand this hint for guidance on signing in to GitHub in Visual Studio Code. 

- On the Windows taskbar, in Search, enter `Visual Studio Code`, and then select **Visual Studio Code**.
- In the bottom right-hand corner, select *Signed out*, select **Sign in to use Copilot**, and then sign in as `<GithubUN>` by using `<GithubPW>` as the password.
- Select **Continue**, and then select **Open** in the pop-up window.

 You may close the *Welcome* tabs.

- Create a new file named `my_script.py` in the `D:\LabFiles` folder.

💡 Expand this hint for guidance on creating a new file. ^

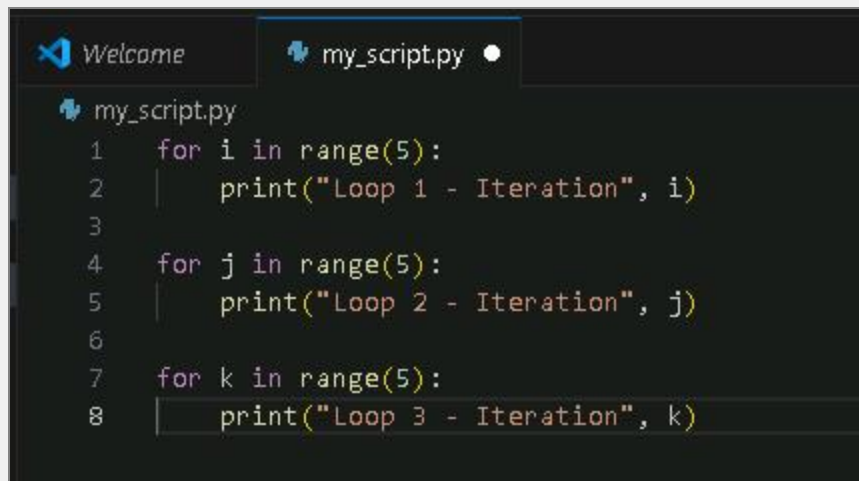
- In **VS Code**, select the Explorer icon, select *Open Folder*, navigate to the `T D:\LabFiles` folder, and then select **Select Folder**.
- On the *Do you trust the authors of the files in this folder?* dialog, select **Yes, I trust the authors**.
- In the Explorer panel, select the *LABFILES* folder, select the **New File** icon, and then enter `T my_script.py`.

💡 Want to learn more? Review the documentation on [creating files in Visual Studio Code](#).

- Use Copilot to generate a starting script containing repetitive loops by using the prompt, `T # Create multiple repetitive loops that we can refactor later`.

💡 Expand this hint for guidance on generating a starting script using Copilot. ^

- At the top of the **my_script.py** file, select **Open chat**, enter the prompt from above, press **Send**, and then pause to allow Copilot to suggest full example loops.
- Select **Accept** to use Copilot's suggestion. It should produce 2-3 loops performing similar tasks (printing items, iterating over lists, etc.). Copilot's suggestion will be similar to the screenshot below.



```
1  for i in range(5):
2      print("Loop 1 - Iteration", i)
3
4  for j in range(5):
5      print("Loop 2 - Iteration", j)
6
7  for k in range(5):
8      print("Loop 3 - Iteration", k)
```

- If no suggestion appears, enter the first line of a simple loop such as `T for i in range(5):`
- Copilot should complete the rest of the loop.
- Save the **my_script.py** file by selecting **File > Save** or by pressing **Ctrl+S**.

 Want to learn more? Review the documentation on

- [Python for statement syntax](#)
- [GitHub Copilot chat](#)

 This task helps you set up the initial code that needs to be refactored in subsequent tasks.

- Refactor the code into functions using Copilot and the following prompt: `T` # Refactor the following loops into reusable functions.

 Expand this hint for guidance on refactoring code into functions using Copilot. 

- In the **my_script.py** file, in **VS Code**, select the entire script, and then add the prompt from above in the **GitHub Copilot Editor Inline Chat (Ctrl+I)**, asking Copilot to refactor the code into functions.
- Accept Copilot's suggestions for function creation.
- Save the **my_script.py** file

 Want to learn more? Review the documentation on [Function definitions](#).

 Make sure Copilot is properly installed and enabled in your IDE for it to provide suggestions.

 Ensure that the moved logic is correctly placed in the functions to avoid breaking the original functionality. 

Test your script thoroughly after refactoring.

Check your work

Verify

Enhance code clarity with type hints and comments

Hints Enabled

No Yes

- Add type hints to functions using GitHub Copilot and the following prompt: `[T] # Add type hints to these functions.`



Expand this hint for guidance on using Copilot to add type hints to functions. ^

- In **VS Code**, in the **my_script.py** file, select the entire script, open **GitHub Copilot Editor Inline Chat (Ctrl+I)**, and enter the prompt from above, asking Copilot to add type hints.
- Wait for Copilot to suggest type hints and accept Copilot's generated code.
- Save the file.



Want to learn more? Review the documentation on [Python type hints](#).



Using correct type hints improves code readability and helps catch potential errors during development.

- Generate docstrings for each function using Copilot and the following prompt: `[T] # Generate a docstring for this function.`

💡 Expand this hint for guidance on generating docstrings for each function using Copilot. ^

- In the **my_script.py** file, locate a function, place your cursor on the line above it, and enter the prompt from above in the **GitHub Copilot Editor Inline Chat** to ask Copilot to generate a docstring.

Note: Copilot should provide suggestions for generating docstrings. Accept these suggestions to automatically create docstrings for each function.

- Ensure that the docstrings accurately describe what the function does, what parameters it takes, and what it returns.
- Save the file.

💡 Want to learn more? Review the documentation on [Python Docstrings](#).

📄 Alternatively, you can manually type the beginning of a docstring (`T """`) and Copilot will often suggest a complete docstring based on the function's purpose and parameters.

❓ Docstrings are essential for documenting your code and making it easier for others (and yourself) to understand. Copilot's suggestions will vary and your results may not look exactly the same.

```
12
13 loop1()
14 loop2()
15 loop3()
16
17 def loop1() -> None:
18     """
19     Iterates over a predefined sequence or performs
20
21     Currently, this function is a placeholder and d
22     """
23     pass
24
25 def loop2() -> None:
26     """
27     Iterates over a predefined sequence or performs
28
29     Currently, this function is a placeholder and d
30     """
31     pass
32
33 def loop3() -> None:
34     """
35     Executes a loop operation three times.
36
37     This function currently does not implement any
38     """
39     pass
```

- Add inline comments to complex logic sections via Copilot suggestions by using the following prompt: `T # Add inline comments to explain this script.`

 Expand this hint for guidance on adding inline comments to complex logic sections via Copilot suggestions. ^

- In the **my_script.py** file, select the entire script, and add inline comments by entering the prompt from above in the **GitHub Copilot Editor Inline Chat**.
- Review and accept Copilot's suggestions for inline comments. These comments should explain what the code is doing and why.
- Save the file.



Want to learn more? Review the documentation on [code commenting](#).

- Create a standard comment block at the top of the **my_script.py** file, which includes the file name, a description of the file, the author, and the date created. This comment should be placed above the import statements.

Python



```
"""  
my_script.py  
  
Examples of repetitive loops broken out into functions.  
  
Author: <Name>  
Created: [Insert Today's Date]  
"""
```



Avoid over-commenting; focus on explaining the *why* rather than the *what* when the code is already clear. ^

Comments should clarify intent, not restate the code.

Check your work

Verify

Improve code efficiency using Copilot's suggestions

Hints Enabled

No Yes

- Use **GitHub Copilot Chat** to get suggestions for simplification of a function by using the following prompt:



How can I simplify this function to make it more efficient? Add a comment that says the function has been simplified.



Expand this hint for guidance on using GitHub Copilot Chat to get suggestions for simplification of functions. ^

- In the **my_script.py** file, in **VS Code**, select one of the functions Copilot generated earlier.
- In **GitHub Copilot Chat**, ask for simplification suggestions by using the prompt from above.
- Review the suggestions provided by **GitHub Copilot Chat**, and then apply the changes.
- Save the file.



Want to learn more? Review the documentation on [GitHub Copilot chat](#).

- In **my_script.py**, press **Ctrl+A**, right-click, select **Copilot**, and then select **Fix**. Copilot's suggestions will vary and your results may not look exactly the same.

```

1  def loop1():
2      for i in range(5):
3          print(f"Loop 1 - Iteration {i}")
4
5  def loop2():
6      # Loop from 0 to 4 (inclusive)
7      for j in range(5):
8          # Print the current iteration number for loop 2
9          print("Loop 2 - Iteration", j)
10
11 def loop3():
12     for k in range(5):
13         print("Loop 3 - Iteration", k)
14
15 loop1()
16 loop2()
17 loop3()
18
19 # Removed duplicate placeholder function definition

```

- Extract a shared helper function for printing loop labels and iteration numbers using GitHub Copilot, and then call the new function using the following prompt:

 Refactor these loops to use a single helper function called `print_loop(label, count)` that handles the for-loop and print logic.

 Expand this hint for guidance on extracting a helper function from the existing loops. 

- In the **my_script.py** file, open **GitHub Copilot Chat**, and then enter the prompt from above.
- Review Copilot's suggestion.
- Accept the suggestion, and then apply the suggestion to the Editor.

 Want to learn more? Review the documentation on [helper functions](#).

- Use **GitHub Copilot Chat** to refactor the example usage into a single loop that iterates over multiple `T (label, count)` pairs by using the following prompt:

 Refactor these repeated `print_loop` calls into a single loop that iterates

over a list of (label, count) pairs.



Expand this hint for guidance on refactoring the example usage into a loop. ^

- In the **my_script.py** file, review the current code, press **Ctrl+A** to select the entire script, open **GitHub Copilot Chat**, and enter the prompt from above.
- Review Copilot's suggestion, and then apply the suggestion to the Editor.



Want to learn more? Review the documentation on [Python lists](#).



F-strings offer a more concise and readable way to format strings in Python.



Ensure that the helper functions are properly generalized to handle different inputs and that they return the correct results.

Check your work

Verify

Summary

Congratulations, you have completed the **Refactor Python Code by Using Copilot** Challenge Lab.

You have accomplished the following:

- Refactored code into reusable functions.
- Enhanced code clarity with type hints and comments.
- Improved code efficiency using Copilot's suggestions.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]
- Refactor Python Code by Using Copilot [Guided]

- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]

